

Trabajo Práctico 1 — Tu primera escena y tu primer juego de texto

Diplomatura de Videojuegos · Semana 1 (clases 1 y 2) Objetivo: en la **Parte A** armás, casi sin código, una escena con **piso, personaje animado y una caja que cae**. En la **Parte B** practicás **variables, arreglos, `if`, `for`, funciones e input** con **una escena por tema**, igual que en clase, y al final los juntás en un combate por consola: *La Cripta del Golem*.

Qué vas a lograr

Parte A · La escena. Al apretar : un **piso** de color, un **personaje animado** parado encima y una **caja** que cae por gravedad y aterriza. Sin escribir código. El objetivo es perderle el miedo al editor y entender cómo se **arma y conecta** una escena.

Parte B · El código. Seis escenas chiquitas, una por concepto, cada una con su script y su salida en el panel **Output**. Y una séptima que los junta:

```
=== LA CRIPTA DEL GOLEM ===
Aria se enfrenta a un Golem de piedra!
Tu arsenal:
  [0] Espada (daño 25)
  [1] Arco (daño 15)
  [2] Hacha (daño 40)
Controles: ← →  cambiar arma | ENTER  atacar  | ↑  poción
---
Aria: 100 HP | Golem de piedra: 120 HP | Pociones: 2
---
```

 **Tiempo estimado:** dos sesiones. **40–50 min** la Parte A y **60–90 min** la Parte B. Leé cada paso completo antes de hacerlo, y **escribí el código vos**: se aprende tecleando y rompiendo.

Cómo va a quedar el proyecto

Todo vive en **un solo proyecto**, `tp1`. La Parte A es una escena; la Parte B son siete escenas, cada una con su script:

```
res://
├─ nivel1.tscn                ← Parte A
├─ 01_variables.tscn         01_variables.gd      ← B.1
├─ 02_arreglos.tscn          02_arreglos.gd       ← B.2
├─ 03_condicionales.tscn     03_condicionales.gd  ← B.3
├─ 04_for.tscn               04_for.gd            ← B.4
├─ 05_funciones.tscn         05_funciones.gd      ← B.5
├─ 06_input.tscn             06_input.gd          ← B.6
└─ 07_cripta.tscn            07_cripta.gd         ← B.7 (el juego)
```

🔗 Si hiciste el proyecto [clase-02](#) en la **teórico-práctica**, ya conocés el ritual de crear una escena con un script. Acá lo repetís en un proyecto nuevo, con los nombres de arriba.

Parte A — Tu primera escena

🧩 Cómo va a quedar el árbol de nodos

Este es el destino. Tenelo a mano y andá comparando a medida que avanzás:

```
Nivel1 (Node2D)                ← raíz de la escena
├─ Piso (RigidBody2D)          ← A.1
│   └─ Sprite2D
│       └─ CollisionShape2D
├─ Jugador (CharacterBody2D) ← A.2
│   └─ AnimatedSprite2D
│       └─ CollisionShape2D
└─ Caja (RigidBody2D)          ← A.3
    └─ Sprite2D
        └─ CollisionShape2D
```

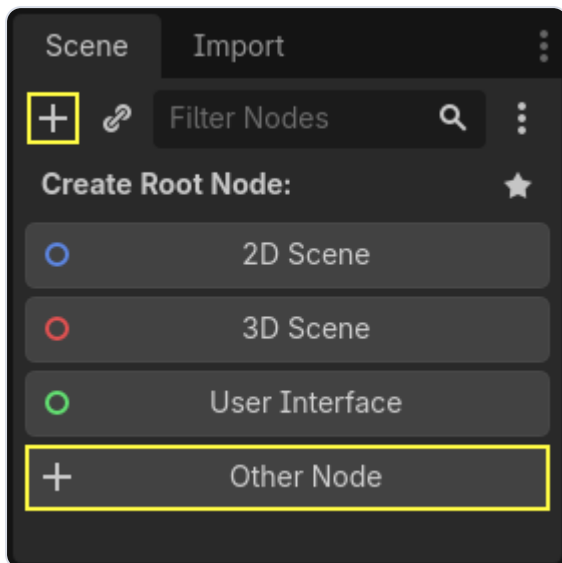
No saltees el orden: cada parte usa lo de la anterior.

🔧 A.0 — Preparar el proyecto

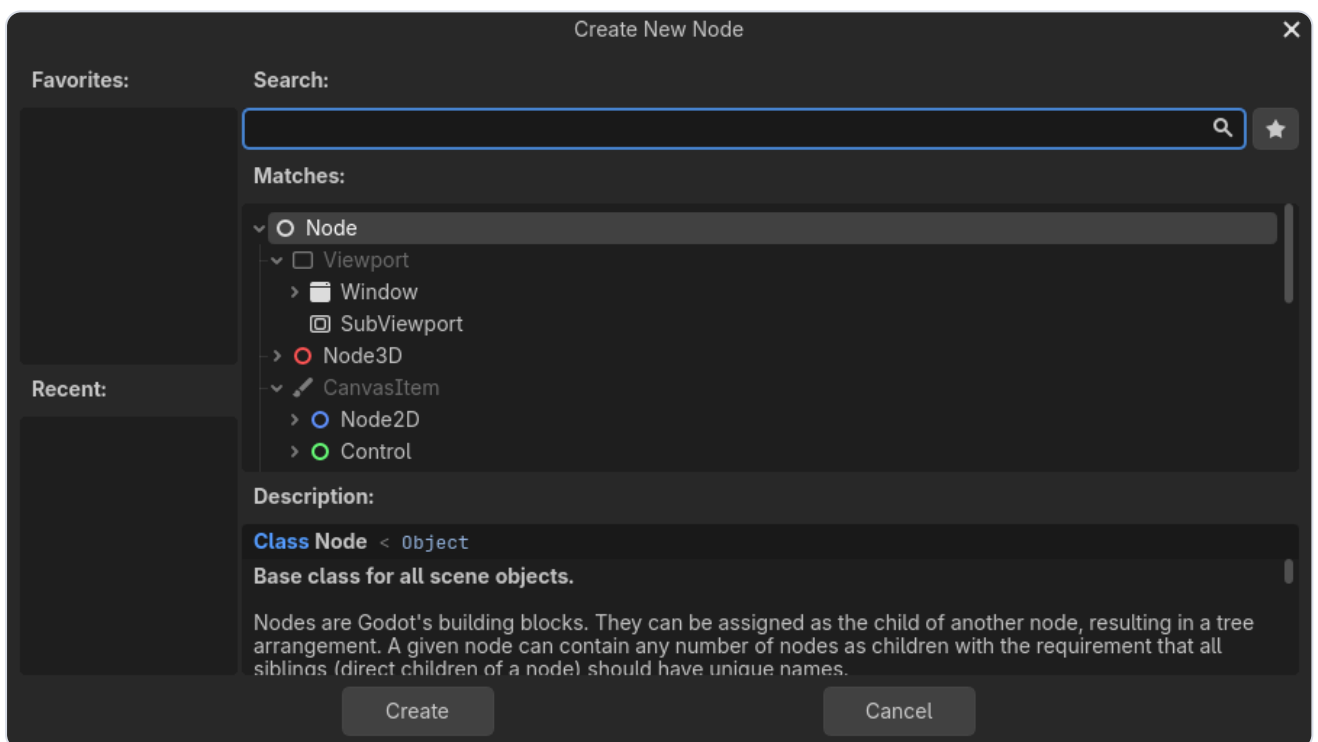
1. Abrí Godot. En el **Project Manager**, clic en **New Project** (Nuevo proyecto).
2. Nombre: `tp1`. Elegí una carpeta **vacía**. Dejá el renderizador en el que viene por defecto → **Create & Edit**.
3. Cuando abre el editor, arriba a la izquierda ves el panel **Escena (Scene)**. Todavía no hay ningún nodo.

Crear la escena y su raíz

4. En el panel **Escena**, entre las opciones rápidas, clic en **Otro Nodo** (*Other Node*).



5. Se abre el buscador de nodos. Escribí `Node2D`, seleccionalo y **Crear**.



6. En el panel Escena, **doble clic** sobre el nombre del nodo y renombralo a `Nivel1`.

Por qué `Node2D` como raíz: es el nodo base de todo lo 2D; aporta posición, rotación y escala en el plano. Todo lo demás va a **colgar** de él, formando el *árbol de escena* que vimos en clase.

7. Guardá con **Ctrl + S**. Nombre del archivo: `nivel1.tscn`. Fijate que aparece en el panel **FileSystem** (abajo a la izquierda). Recordá: las rutas del proyecto empiezan con `res://`.

✓ **Punto de control A.0:** tenés un único nodo `Nivel1 (Node2D)` en el árbol y un archivo `nivel1.tscn` guardado.

■ A.1 — El Piso (RigidBody2D)

Conceptos: importar un sprite · modular su color · gravedad y cómo desactivarla.

A.1.1 · Crear el nodo del piso

1. Seleccioná `Nivel1` en el árbol.
2. Clic en el botón **+ Agregar Nodo Hijo** (*Add Child Node*), el **+** arriba a la izquierda del panel Escena, o el atajo **Ctrl + A**. Se abre el mismo buscador de nodos de antes.
3. Buscá `RigidBody2D`, crealo, y renombralo a `Piso`.

💡 **¿Qué es un RigidBody2D?** Un cuerpo con **física real**: le afecta la gravedad, choca, rebota. Lo usamos acá **a propósito** para ver la gravedad en acción y después apagarla. (Ojo: un piso “de verdad” normalmente sería un `StaticBody2D`; lo vemos como dato al final de esta parte.)

A.1.2 · Darle imagen: el Sprite2D

4. Con `Piso` seleccionado, agregá un hijo (**Ctrl + A**) → `Sprite2D`.
5. Necesitamos una imagen. Usá la que **ya viene con el proyecto**: en el panel **FileSystem** vas a ver `icon.svg`.
6. **Arrastrá** `icon.svg` desde FileSystem hasta la propiedad **Texture** del Sprite2D en el **Inspector** (derecha). Aparece el logo de Godot en el centro.

💡 **Importar = poner el archivo en la carpeta del proyecto.** Godot lo detecta e importa solo. Si más adelante querés otra imagen, copió el archivo dentro de la carpeta del proyecto (o arrastralo al FileSystem) y listo.

A.1.3 · Convertirlo en plataforma y cambiarle el color

7. Con el **Sprite2D** seleccionado, en el Inspector buscá **Transform** → **Scale** y poné `x = 6`, `y = 0.6`. El logo se estira: ahora parece una plataforma ancha y finita.
8. Cambiá el color: en el Inspector, abrí **Visibility** → **Modulate** (dentro de la sección **CanvasItem**). Clic en el rectángulo de color y elegí un color, por ejemplo un verde.

💡 `Modulate` es un **tinte**. Multiplica el color del sprite por el que elijas: no “pinta” la imagen, la **tiñe**. Es la forma más rápida de dar variedad sin abrir un editor de imágenes.

A.1.4 · Darle cuerpo físico: el CollisionShape2D

9. Seleccioná **Piso** de nuevo → agregá hijo (**Ctrl + A**) → **CollisionShape2D**.

⚠ Si aparece un **triángulo amarillo** ⚠ al lado del nodo, es normal: te avisa que **todavía no tiene forma**. Lo arreglamos ahora.

10. Con el **CollisionShape2D** seleccionado, en el Inspector buscá la propiedad **Shape** y hacé clic en **<vacío>** → **Nuevo RectangleShape2D**.
11. En el Viewport aparecen unos **puntos naranjas**: arrastralos hasta que el rectángulo **cubra la plataforma verde**. No tiene que ser perfecto, pero que tape bien la parte de arriba.

💡 El **Sprite** es lo que se ve; el **CollisionShape** es lo que se toca. Son dos cosas separadas a propósito: podés tener una imagen enorme con una colisión chiquita, o al revés. Si no ponés forma, **nada choca**.

A.1.5 · Ver la gravedad... y apagarla

12. Posicioná el piso: seleccioná **Piso** y, en el Inspector (**Transform** → **Position**), poné algo como **x = 0**, **y = 300** (recordá: en 2D, **la Y crece hacia abajo**).
13. Apretá **▶ (Play Scene)** arriba a la derecha, o **F6**.



Si te pide **guardar** o elegir escena principal, guardá **nivel1.tscn**.

14. ¿Qué pasa? El piso **se cae de la pantalla** 😬. Es correcto: un **RigidBody2D** tiene gravedad. Cerrá la ventana del juego.
15. Ahora **apagá la gravedad**: seleccioná **Piso**, en el Inspector buscá la sección **RigidBody2D** y poné **Gravity Scale = 0**.
16. Además, para que quede **clavado como piso sólido** (que la caja no lo empuje), activá **Freeze** (Congelar). Si aparece **Freeze Mode**, dejalo en **Static**.
17. Volvé a apretar **▶**. Ahora el piso **se queda quieto**. 🎉

💡 Dos propiedades distintas:

- **Gravity Scale** = cuánta gravedad le afecta. En **0**, no cae.
- **Freeze** = lo "congela" en su lugar, inmóvil, aunque algo lo golpee.

💡 **Dato:** para un piso real que nunca se mueve conviene un **StaticBody2D** (no necesita nada de esto). Usamos **RigidBody2D** solo para *entender* la gravedad. En la caja (A.3) la vamos a dejar **encendida** para verla caer.

- ✓ **Punto de control A.1:** al dar Play, ves una plataforma verde **quieta** en la parte de abajo.

A.2 — El Jugador (CharacterBody2D + animación)

Conceptos: `CharacterBody2D` · `AnimatedSprite2D` con `SpriteFrames` · `CollisionShape2D` .

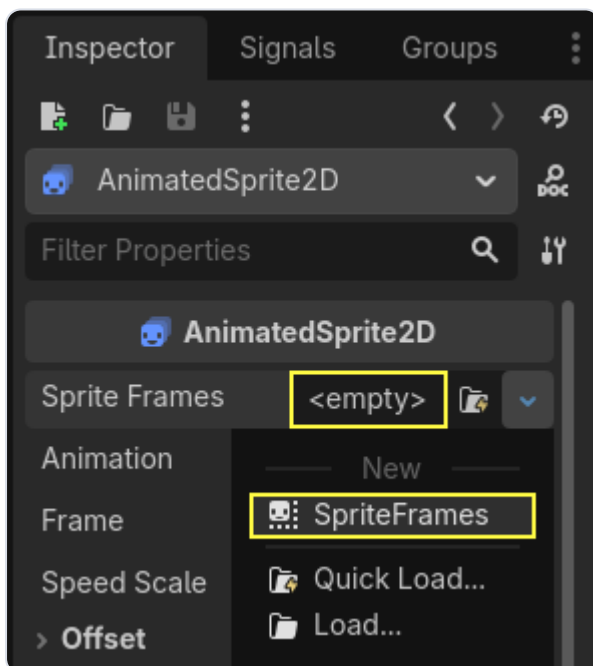
A.2.1 · El nodo del jugador

1. Seleccioná `Nivel1` → agregá hijo (**Ctrl + A**) → `CharacterBody2D` → renombralo `Jugador` .

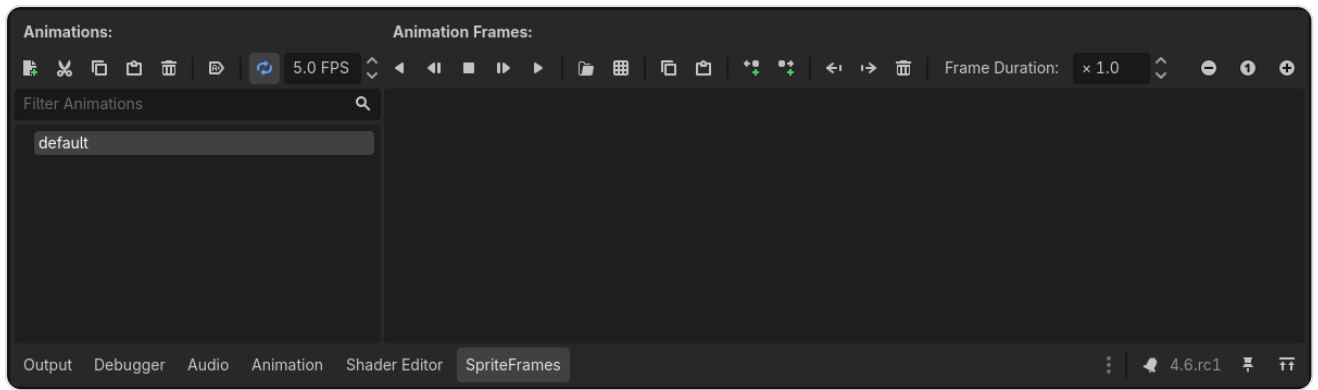
`CharacterBody2D` es el nodo pensado para **personajes controlados**. A diferencia del `RigidBody2D` , **no se mueve solo**: se mueve por código. Por eso hoy va a quedar **flotando quieto** donde lo pongas. La semana que viene lo hacemos caminar.

A.2.2 · La animación: `AnimatedSprite2D` + `SpriteFrames`

2. Con `Jugador` seleccionado, agregá hijo → `AnimatedSprite2D` .
3. En el Inspector, buscá la propiedad **Sprite Frames**, clic en `<vacío>` → **Nuevo `SpriteFrames`**.



4. **Clic sobre el recurso `SpriteFrames`** que acabás de crear: abajo se abre el panel **`SpriteFrames`**.



5. Ahí vas a cargar los **cuadros** de la animación. Tenés dos caminos:

Camino A — con un sprite sheet (recomendado):

- Descargá un personaje gratis (CC0) de kenney.nl/assets (por ejemplo “Pixel Platformer” o “Platformer Characters”).
- Copiá las imágenes dentro de la carpeta del proyecto (arrastralas al **FileSystem**).
- En el panel SpriteFrames, usá **+** (**Añadir cuadros desde un archivo**) y elegí las imágenes de la animación de caminar o de idle. Se van agregando como cuadros.

Camino B — sin descargar nada (plan B):

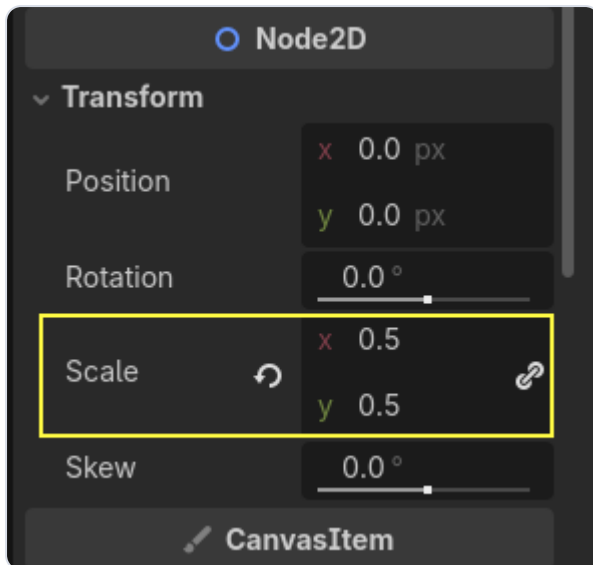
- En el panel SpriteFrames, agregá **el mismo** `icon.svg` **dos veces** como dos cuadros.
- Dejá los dos cuadros iguales o cambiale el `Modulate` a uno: la idea es **ver el sistema funcionar**, no que sea lindo.

6. Renombrá la animación (arriba en el panel) a `idle` si querés, y **subí los FPS** a **~6** para que se note.

7. **Que arranque sola y en loop:** en el panel SpriteFrames activá el botón **Loop** y el botón **Autoplay on Load** (el ícono con una **A**). Así la animación se reproduce **sin código**.

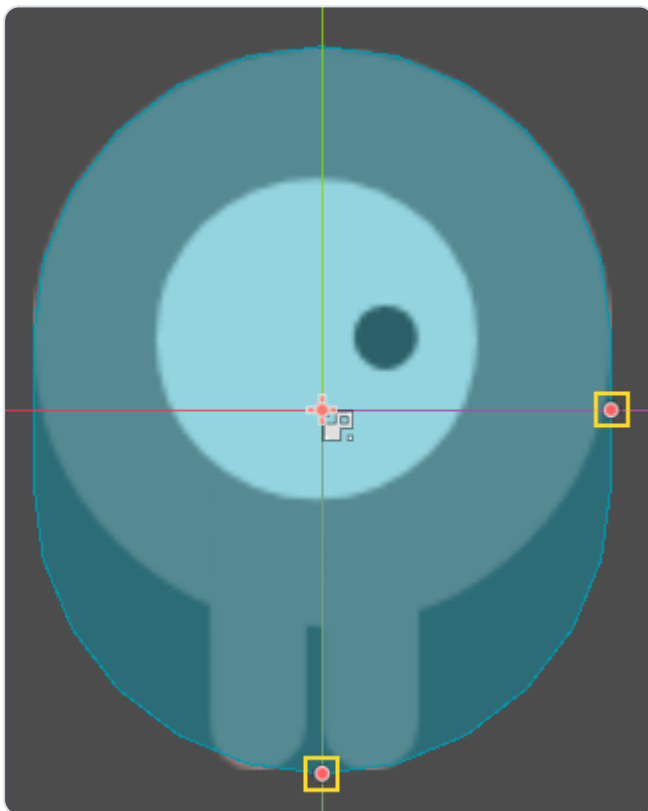
SpriteFrames es un recurso que guarda **listas de cuadros** (una por animación: idle, correr, saltar...). El **AnimatedSprite2D** es el nodo que las **muestra**. Es el equivalente animado del **Sprite2D**.

8. Si el personaje se ve gigante, seleccioná el **AnimatedSprite2D** y bajá **Transform** → **Scale** a algo como `0.5, 0.5`.



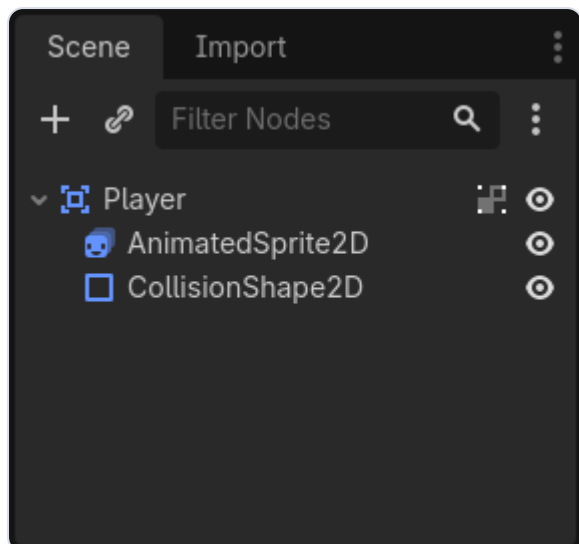
A.2.3 · Su colisión

9. Seleccioná **Jugador** → agregá hijo → **CollisionShape2D**.
10. En el Inspector → **Shape** → **<vacío>** → **Nuevo CapsuleShape2D** (una cápsula va bien para un cuerpo). Ajustá los puntos naranjas para que **envuelva** al personaje.



11. Posicioná al **Jugador** **arriba del piso** (por ejemplo **Position** **$x = 0$** , **$y = 0$**).

✓ **Punto de control A.2:** al dar ▶, ves al personaje **animándose** (aunque quieto en el lugar). Debería quedar parecido a esto:



A.3 — La Caja que cae (RigidBody2D)

Conceptos: `RigidBody2D` con **gravedad encendida** · `Sprite2D` · `CollisionShape2D` chocando contra el piso.

1. Seleccioná `Nivel1` → agregá hijo → `RigidBody2D` → renombralo `Caja`.
2. Hijo de `Caja` → `Sprite2D` → arrastrale `icon.svg` a **Texture**. Bajale la escala a algo chico, tipo `0.4`, `0.4`. (Opcional: `Modulate` marrón para que parezca caja.)
3. Hijo de `Caja` → `CollisionShape2D` → **Shape** → **Nuevo RectangleShape2D** → ajustá el rectángulo al tamaño del sprite.
4. Posicioná la `Caja` **bien arriba y sobre el piso**: por ejemplo **Position** `x = 150`, `y = -200` (un poco a la derecha, así no cae sobre el jugador).
5. **Importante:** esta vez **NO** toques `Gravity Scale` ni `Freeze`. Queremos que **caiga**.
6. Apretá . La caja **cae por gravedad** y **atteriza sobre la plataforma verde**. 🇺🇵

💡 Esto es lo que hace un motor por vos: **no programaste la caída ni el choque**. El `RigidBody2D` + los `CollisionShape2D` resuelven la física solos. Sin motor, esto serían muchas horas de matemática.

 ¿La caja atraviesa el piso o no aparece?

▼ Abrió para ver soluciones

- ¿El **piso** tiene su `CollisionShape2D` con **forma** asignada? Sin forma, no choca.
- ¿La **caja** tiene su `CollisionShape2D` con forma?
- ¿La caja arranca **arriba** del piso (Y más chica) y **dentro** del ancho de la plataforma?
- ¿Dejaste el piso con `Freeze = On` ? Si no, la caja lo empuja hacia abajo.

✓ **Punto de control A.3:** la caja cae y **se queda apoyada** sobre el piso, al lado del personaje animado. **Guardá con Ctrl + S.** ¡Terminaste la Parte A! 🎉

🔭 **Lo que falta a propósito:** cámara, música y el cartel de UI llegan en las próximas semanas, cuando el personaje ya se mueva. Si te sobra tiempo, está en los extras.

Parte B — Tu primer juego de texto

💬 Antes de arrancar: ¿por qué “en la consola”?

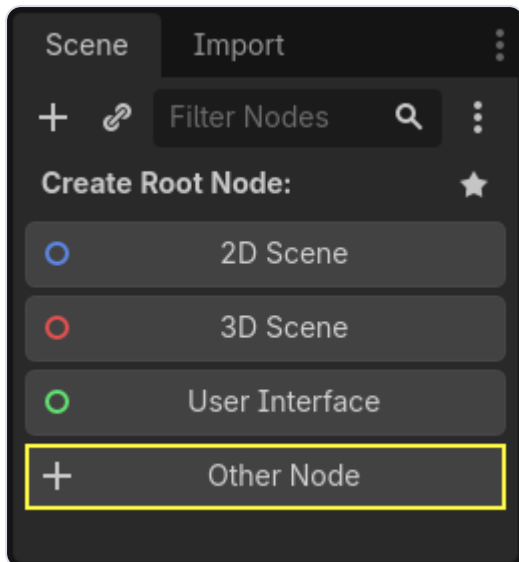
En la clase vimos que `print()` es nuestro mejor amigo para depurar. Acá lo usamos como *pantalla*: en vez de mover un personaje, vamos a **imprimir** lo que pasa. Así nos concentramos en la **lógica** (variables, decisiones, repeticiones) sin pelear con gráficos.

Lo único “gráfico” va a ser el **input**: leemos las teclas con el sistema `Input` de Godot, pero la imagen del juego es el texto que aparece en el panel **Output**.

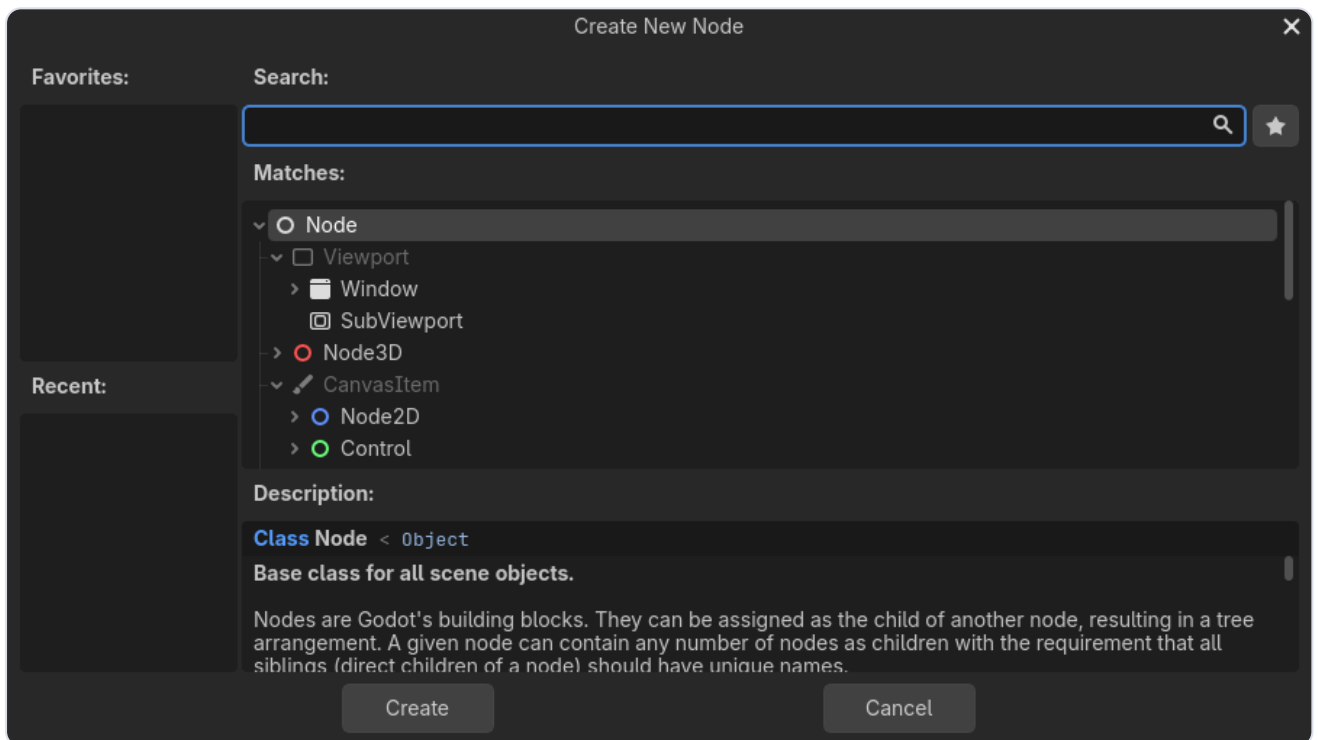
🔄 B.0 — El ritual: una escena, un nodo, un script

Cada parte de acá en adelante arranca **igual**, con estos cinco pasos. Los hacemos juntos la primera vez; después el TP solo te va a decir “*el ritual, con tal nombre*”.

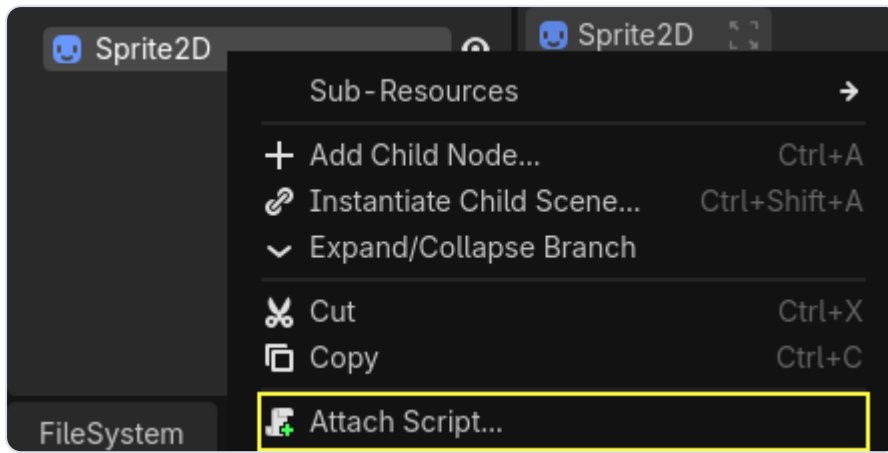
1. En el mismo proyecto `tp1`, menú **Scene** → **New Scene** (Ctrl + N). En el panel **Escena**, clic en **Otro Nodo** (*Other Node*).



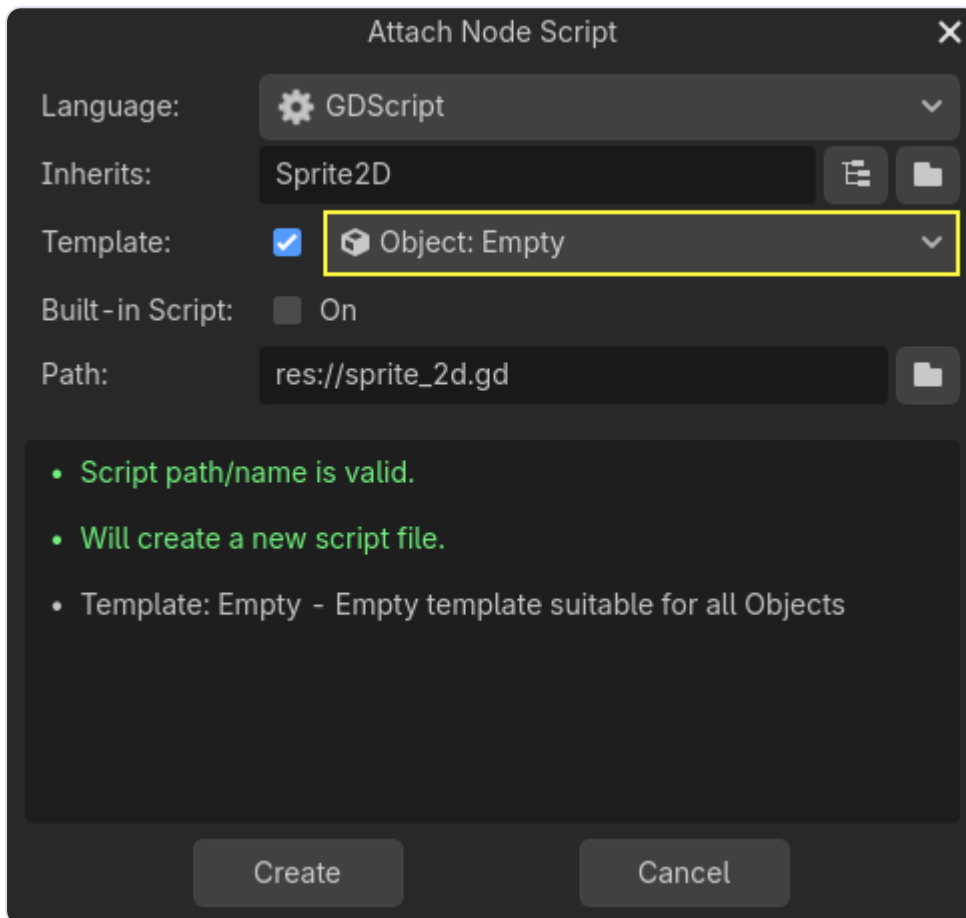
2. Buscá **Node** (el nodo más básico, sin nada visual), selecciónalo y **Crear**.



3. Renombralo (doble clic sobre el nombre). En esta primera escena: **Variables**.
4. **Adjuntá un script**: clic derecho sobre el nodo → **Attach Script** (*Adjuntar Script*), o el botón con el ícono  arriba del panel Escena.



5. En el diálogo, dejá **Language: GDScript** y cambiá el **Path** a `res://01_variables.gd`. Clic en **Create**.



6. Se abre el **editor de Script** con una plantilla. **Borrá todo** y dejá solo esto:

```
extends Node

func _ready():
    print("¡La escena arrancó!")
```

7. Guardá con **Ctrl + S**. Te pide el nombre de la escena: `01_variables.tscn`.
8. **Ejecutá la escena actual** con **F6** (no F5: F5 corre la escena principal del proyecto, que es `nivel1`). Mirá el panel inferior **Output** (*Salida*): tiene que aparecer tu mensaje.

```
Godot Engine v4.6.rc1.official
D3D12 12_0 - Forward+ - Using
1060 6GB

Hello, world!
--- Debugging process stopped
```

 `extends Node` dice qué tipo de nodo controla el script. `func _ready():` se ejecuta **una sola vez** al aparecer el nodo. Todo lo que imprimas ahí aparece en **Output**. Y si al correr se abre una ventana gris y vacía, **está bien**: el juego “es” lo que sale en la consola.

✓ **Punto de control B.0:** ves “¡La escena arrancó!” en el panel Output al apretar F6, y en el FileSystem están `01_variables.tscn` y `01_variables.gd`.

No aparece nada / da error

▼ Abrió para ver soluciones

- La **sangría importa**: lo de adentro de `func` va corrido con **un Tab**. No mezcles tabs y espacios.
- ¿Guardaste con **Ctrl + S** antes de apretar F6?
- ¿Apretaste **F6** (escena actual) y no F5?
- ¿El panel de abajo está en la pestaña **Output**, no en **Debugger**?

B.1 — Variables: la ficha de la heroína

Escena: `01_variables` (la que acabás de crear en B.0). **Concepto:** asignación de variables (`var`) y tipos (`int` , `float` , `String` , `bool`), imprimir con `str()` .

Reemplazá el `func _ready()` de `01_variables.gd` por esto y apretá **F6**:

```
extends Node

func _ready():
    var nombre = "Aria"          # String  (texto)
    var vida = 100                # int      (entero)
    var precision = 85.5          # float   (decimal)
    var es_heroína = true         # bool    (verdadero/falso)

    print("Nombre: " + nombre)
    print("Vida: " + str(vida))
    print("Precisión: " + str(precision) + "%")
    print("¿Es heroína?: " + str(es_heroína))
```

Deberías ver en Output:

Nombre: Aria
Vida: 100
Precisión: 85.5%
¿Es heroína?: true

🗨️ Una variable es una **caja con etiqueta**: `vida` guarda un `100`. Para **pegar** un número con texto hay que convertirlo con `str()`: `"Vida: " + str(vida)`. Si te olvidás el `str()`, Godot se queja en rojo: no sabe sumar texto con número.

🔧 **Probá vos:** cambiá los valores. Poné tu propio nombre y una vida distinta. Sacale el `str()` a `vida`, corré, y leé el error: lo vas a ver mil veces.

✅ **Punto de control B.1:** imprimís una ficha con los 4 tipos de datos.

📦 B.2 — Arreglos: el arsenal

Escena: el ritual, con nodo `Arreglos`, script `02_arreglos.gd` y escena `02_arreglos.tscn`. **Concepto:** arreglos (arrays): crear, acceder por **índice**, tamaño con `.size()`, agregar con `.append()`.

Un **arreglo** es una variable que guarda **varias cosas en orden**. Cada elemento tiene un **número de posición** (índice) que **empieza en 0**.

```
extends Node

func _ready():
    var armas = ["Espada", "Arco", "Hacha"]

    print("Primer arma: " + armas[0])    # índice 0 → Espada
    print("Segunda arma: " + armas[1])   # índice 1 → Arco
    print("Cuántas armas tengo: " + str(armas.size()))

    armas.append("Daga")                 # agrega al final
    print("Nueva arma agregada: " + armas[3])
    print("Ahora tengo: " + str(armas.size()) + " armas")
```

Output:

```
Primer arma: Espada
Segunda arma: Arco
Cuántas armas tengo: 3
Nueva arma agregada: Daga
Ahora tengo: 4 armas
```

🗨️ **El índice empieza en 0.** En un arreglo de 3 elementos, las posiciones son `0`, `1` y `2`. Pedir `armas[3]` **antes** de agregar la Daga daría error ("índice fuera de rango"): esa posición todavía no existe.

💡 **Truco de diseño: arreglos en paralelo.** Guardamos el **daño** de cada arma en **otro** arreglo, en el mismo orden. `armas[2]` es "Hacha" y `danos[2]` es su daño. Los vamos a usar así en el juego final. Agregalo al final de tu `_ready()`:

```
var danos = [25, 15, 40, 10]
print(armas[2] + " hace " + str(danos[2]) + " de daño") # Hacha hace 40 de daño
```

✅ **Punto de control B.2:** accedés a elementos por índice y usás `.size()` y `.append()`.

🔗 B.3 — Condicionales: decidir el resultado

Escena: el ritual, con nodo `Condicionales`, script `03_condicionales.gd` y escena `03_condicionales.tscn`. **Concepto:** `if` / `elif` / `else` y operadores de comparación (`==`, `!=`, `<`, `>`, `<=`, `>=`).

```
extends Node

func _ready():
    var vida_enemigo = 40
    var dano = 25

    vida_enemigo = vida_enemigo - dano
    print("Vida del enemigo: " + str(vida_enemigo))

    if vida_enemigo <= 0:
        print("💀 El enemigo cayó")
    elif vida_enemigo <= 20:
        print("🔴 El enemigo está por morir")
    else:
        print("🟢 El enemigo sigue fuerte")
```

Output (con estos valores):

```
Vida del enemigo: 15
🔴 El enemigo está por morir
```

🗨️ Godot revisa las condiciones **de arriba hacia abajo** y ejecuta **la primera que sea verdadera**. Con `vida_enemigo = 15` : no es `<= 0` , sí es `<= 20` → imprime la de vida crítica y **saltea el resto**.

🔧 **Probá vos:** cambiá `dano` a `50` (¿qué rama sale?) y a `10` (¿y ahora?). Predecí el resultado **antes** de apretar F6.

✅ **Punto de control B.3:** según el daño, cae en una rama distinta.

🔄 B.4 — `for` : repetir sin copiar y pegar

Escena: el ritual, con nodo `For` , script `04_for.gd` y escena `04_for.tscn` . **Concepto:** el loop `for` , con `range()` y recorriendo un arreglo.

```
extends Node

func _ready():
    # 1) Repetir N veces con range()
    print("--- Aparecen enemigos ---")
    for i in range(3):
        print("Golem #" + str(i + 1) + " entró a la cripta")

    # 2) Recorrer un arreglo elemento por elemento
    print("--- Tu arsenal ---")
    var armas = ["Espada", "Arco", "Hacha"]
    for arma in armas:
        print("Tenés: " + arma)

    # 3) Recorrer con el índice (para arreglos en paralelo)
    var danos = [25, 15, 40]
    for i in range(armas.size()):
        print "[" + str(i) + " ] " + armas[i] + " → daño " + str(danos[i]))
```

Output:

```
--- Aparecen enemigos ---  
Golem #1 entró a la cripta  
Golem #2 entró a la cripta  
Golem #3 entró a la cripta  
--- Tu arsenal ---  
Tenés: Espada  
Tenés: Arco  
Tenés: Hacha  
[0] Espada → daño 25  
[1] Arco → daño 15  
[2] Hacha → daño 40
```

💡 Hay dos formas de recorrer: `for arma in armas` te da directo cada valor ("Espada" , "Arco" ...). `for i in range(armas.size())` te da el **número de posición** (0 , 1 , 2), útil cuando necesitás el índice para leer **dos arreglos en paralelo** al mismo tiempo.

⚠️ **Ojo (de la clase):** en Godot los `for` **no** se usan para la lógica en tiempo real (para eso está `_process()`). Sirven para **inicializar**, generar contenido y recorrer datos, justo lo que hicimos acá.

🔧 **Probá vos:** `range(10)` . Agregá "Arco largo" a `armas` : ¿hace falta tocar el `for` ? ¿Y el tercer `for` sigue andando? (Pista: `danos` quedó más corto.)

✅ **Punto de control B.4:** imprimís listas con `for` , con y sin índice.

🧰 B.5 — Funciones: escribir una vez, usar mil

Escena: el ritual, con nodo `Funciones` , script `05_funciones.gd` y escena `05_funciones.tscn` . **Concepto:** crear funciones, pasarles **parámetros** y devolver un valor con `return` .

```

extends Node

var vida = 100  # variable "global" del script: la ven todas las funciones

func _ready():
    mostrar_estado()
    recibir_dano(30)    # le pasamos un parámetro
    recibir_dano(50)
    if esta_vivo():      # usamos lo que devuelve
        print("Aria sigue en pie")
    else:
        print("Aria cayó")

# Recibe un parámetro y modifica la vida
func recibir_dano(cantidad):
    vida = vida - cantidad
    print("Recibí " + str(cantidad) + " de daño. Vida: " + str(vida))

# Devuelve un valor (true o false)
func esta_vivo():
    return vida > 0

func mostrar_estado():
    print("=== Vida actual: " + str(vida) + " ===")

```

Output:

```

=== Vida actual: 100 ===
Recibí 30 de daño. Vida: 70
Recibí 50 de daño. Vida: 20
Aria sigue en pie

```

Anatomía:

- `func recibir_dano(cantidad):` → **definición**. `cantidad` es el **parámetro** (el dato que entra).
- `recibir_dano(30)` → **llamado**. Los `()` son **obligatorios**.
- `return vida > 0` → **devuelve** un resultado (acá `true` o `false`) que podés usar en un `if`.

Ojo: `vida` está declarada **afuera** de las funciones (arriba de todo). Por eso **todas** la pueden leer y modificar. Es el "estado" del juego.

 **Probá vos:** un tercer `recibir_dano(30)` : ¿qué imprime el `if` ? Escribí `curar(cantidad)` que suma vida y llamala.

 **Punto de control B.5:** definís funciones con parámetro y con `return` , y las llamás.



B.6 — Input: que reaccione a las teclas

Escena: el ritual, con nodo `Input`, script `06_input.gd` y escena `06_input.tscn`. **Concepto:** detectar input con `Input.is_action_just_pressed(...)` dentro de `_process()`.

Hasta ahora todo corría solo en `_ready()`. Para **reaccionar al jugador** necesitamos `_process()`, que corre en **cada frame** (~60 por segundo).

```
extends Node

func _ready():
    print("Presioná: ENTER (atacar), ← →, o ↑")

func _process(delta):
    if Input.is_action_just_pressed("ui_accept"):    # Enter / Espacio
        print("¡Ataque!")
    if Input.is_action_just_pressed("ui_right"):    # flecha →
        print("Elegiste el arma de la derecha")
    if Input.is_action_just_pressed("ui_left"):     # flecha ←
        print("Elegiste el arma de la izquierda")
    if Input.is_action_just_pressed("ui_up"):       # flecha ↑
        print("Tomaste una poción")
```

Apretá **F6**. **Importante:** hacé **clic sobre la ventana del juego** para que reciba las teclas; los mensajes aparecen en el panel **Output** del editor.

 `is_action_just_pressed` se dispara **una sola vez** por pulsación (ideal para menús y ataques). `is_action_pressed` es **true** **mientras** la tenés apretada (ideal para movimiento continuo). Las acciones `ui_accept`, `ui_left`, `ui_right`, `ui_up`, `ui_down`, `ui_cancel` **ya vienen** definidas; podés crear las tuyas en `Project → Project Settings → Input Map`.

 **Probá vos:** cambiá el `just_pressed` del ataque por `pressed` y mantené apretado ENTER. Eso es `_process()`: 60 veces por segundo.

 **Punto de control B.6:** cada tecla imprime su mensaje en Output.

 **Las teclas no hacen nada**

▼ Abrí para ver soluciones

- ¿Hiciste **clic en la ventana del juego**? Si el foco está en el editor, no llegan las teclas.
- El código de input va en `_process(delta)`, no en `_ready()`.
- Revisá que escribiste `ui_accept` (con guion bajo), entre comillas.



B.7 — Todo junto: *La Cripta del Golem*

Escena: el ritual, con nodo `Cripta` , script `07_cripta.gd` y escena `07_cripta.tscn` .

Ahora combinamos **las seis piezas** en un solo juego. Escribí esto leyéndolo: cada bloque dice **qué parte del TP usa** (están marcados con comentarios).

```

extends Node

# ---- VARIABLES: estado del juego (B.1) ----
var nombre = "Aria"
var vida = 100
var pociones = 2
var terminado = false

# ---- ARREGLOS en paralelo (B.2) ----
var armas = ["Espada", "Arco", "Hacha"]
var danos = [25, 15, 40]
var arma_actual = 0          # índice del arma elegida

# ---- Enemigo ----
var enemigo = "Golem de piedra"
var vida_enemigo = 120

func _ready():
    print("=== LA CRIPTA DEL GOLEM ===")
    print(nombre + " se enfrenta a un " + enemigo + "!")
    mostrar_inventario()      # usa un FOR (B.4)
    print("Controles: ← → cambiar arma | ENTER atacar | ↑ poción")
    mostrar_estado()

func _process(delta):
    # INPUT (B.6). Si el juego terminó, no hacemos nada.
    if terminado:
        return

    if Input.is_action_just_pressed("ui_right"):
        arma_actual = arma_actual + 1
        if arma_actual >= armas.size(): # si me pasé, vuelvo al principio
            arma_actual = 0
        print("Arma seleccionada: " + armas[arma_actual])

    if Input.is_action_just_pressed("ui_left"):
        arma_actual = arma_actual - 1
        if arma_actual < 0:
            arma_actual = armas.size() - 1
        print("Arma seleccionada: " + armas[arma_actual])

    if Input.is_action_just_pressed("ui_accept"):
        atacar()

    if Input.is_action_just_pressed("ui_up"):
        curar()

# ---- FUNCIONES (B.5) ----

func mostrar_inventario():

```

```

print("Tu arsenal:")
for i in range(armas.size()):          # FOR con índice
    print("  [" + str(i) + "] " + armas[i] + " (daño " + str(danos[i]) + ")")

func mostrar_estado():
    print("---")
    print(nombre + ": " + str(vida) + " HP   |   " + enemigo + ": " + str(vida_enemigo) + " HP
    | Pociones: " + str(pociones))
    print("---")

func atacar():
    var dano = danos[arma_actual]      # leo el arreglo por índice
    print(nombre + " ataca con " + armas[arma_actual] + " (" + str(dano) + " de daño)")
    vida_enemigo = vida_enemigo - dano

    if vida_enemigo <= 0:              # CONDICIONAL (B.3)
        print("🏆 ¡Derrotaste al " + enemigo + "! GANASTE.")
        terminado = true
        return

    enemigo_contraataca()
    mostrar_estado()

func enemigo_contraataca():
    var golpe = 20
    print("El " + enemigo + " contraataca: -" + str(golpe) + " HP")
    vida = vida - golpe
    if vida <= 0:
        print("💀 " + nombre + " cayó. GAME OVER.")
        terminado = true

func curar():
    if pociones <= 0:
        print("No te quedan pociones.")
        return
    pociones = pociones - 1
    vida = vida + 30
    print(nombre + " bebe una poción: +30 HP")
    mostrar_estado()

```

Cómo se juega: apretá **F6**, hacé **click en la ventana del juego**, y usá ← → para elegir arma, **ENTER** para atacar y ↑ para curarte. Mirá el combate desarrollarse en el panel **Output**.

🗨 **return solo, sin valor**, corta la función ahí mismo. En `_process()` lo usamos para que, cuando el juego terminó, no siga leyendo teclas. En `curar()`, para no restar pociones que no tenés.

✅ **Punto de control B.7 (final):** podés ganarle al Golem eligiendo el Hacha y atacando, o perder si te descuidás. Todo se ve en Output.

🚫 **Errores comunes al armar el juego final**

▼ Abrió para ver soluciones

- “Invalid index / índice fuera de rango”: `armas` y `danos` tienen que tener **la misma cantidad** de elementos.
- **Sangría mezclada**: usá Tab para todo el archivo, no lo combines con espacios.
- **Se repite el ataque sin parar**: usá `is_action_just_pressed` (una vez por pulsación), no `is_action_pressed`.
- **No pasa nada al apretar teclas**: clic en la ventana del juego para darle foco.
- “Identifier not found: atacar”: la función está mal escrita o le falta el `func` adelante. Los nombres tienen que coincidir letra por letra.

Entrega

Entregá **una** de estas opciones (según indique el/la docente):

1. La **carpeta del proyecto** `tp1` comprimida en `.zip` (sin la carpeta `.godot/`), o
2. **Tres capturas**: el `nivel1` corriendo con la caja apoyada, el **árbol de nodos** de `nivel1` (panel Escena), y el panel **Output** mostrando una partida de la Cripta (un ataque, una curación y el final).

Nombre del archivo: `tp1-ApellidoNombre.zip`

✓ Checklist de autoevaluación

Parte A

- ☐ La raíz es un `Node2D` llamado `Nivel1` y guardaste `nivel1.tscn`.
- ☐ El **Piso** es `RigidBody2D`, tiene `Sprite2D` (con color por `Modulate`) y `CollisionShape2D`, y **no se cae** (`Gravity Scale = 0` + `Freeze`).
- ☐ El **Jugador** es `CharacterBody2D` con `AnimatedSprite2D` (animación que **se reproduce sola**) y `CollisionShape2D`.
- ☐ La **Caja** es `RigidBody2D`, **cae** y **aterriza** sobre el piso.

Parte B

- ☐ Tenés **siete escenas** (`01_variables` a `07_cripta`), cada una con un `Node` y su script adjunto, y todas imprimen en **Output**.
- ☐ Usás **variables** de al menos 3 tipos distintos (`int`, `String`, `bool` ...).
- ☐ Usás un **arreglo** y accedés a un elemento por **índice**.
- ☐ Usás `if` / `elif` / `else` para decidir un resultado.
- ☐ Usás un `for` para recorrer el arsenal.
- ☐ Definiste al menos **2 funciones** (una con **parámetro**, una con `return`).
- ☐ El juego reacciona a **teclas** con `Input.is_action_just_pressed`.

- ☐ Se puede **ganar** y **perder** el combate.

☀ Extra (opcional, para los que quieran más)

Sobre la escena (Parte A)

- **Más cajas:** duplicá la `Caja` (**Ctrl + D**) y movelas: vas a ver la física apilándolas.
- **Piso inclinado:** rotá el `Piso` unos grados (**Transform** → **Rotation**) y mirá cómo la caja **resbala**.
- **Cámara:** agregá un `Camera2D` como **hijo del** `Jugador` y tildá `Enabled`. Como cuelga del jugador, cuando él se mueva (la semana que viene) la cámara lo va a seguir sola.
- **Música:** conseguí un `.ogg` o `.mp3` corto (CC0 en kenney.nl/assets), copialo al proyecto, agregá un `AudioStreamPlayer` hijo de `Nivel1`, arrastrá el archivo a **Stream** y tildá **Autoplay**.

Sobre el juego (Parte B)

- **Daño variable (azar):** hacé que cada golpe varíe un poco. `randi_range(min, max)` devuelve un entero al azar:

```
var dano = danos[arma_actual] + randi_range(-5, 5)
```

- **Segundo enemigo:** guardá los enemigos en arreglos en paralelo (`enemigos = [...]` , `vidas_enemigo = [...]`) y pasá al siguiente con un índice cuando cae uno.
- **Función** `esta_vivo()` que devuelva `vida > 0` y usala en los `if` en vez de comparar a mano.
- **Estadísticas al final:** contá cuántos turnos duró el combate con una variable `turnos` que suba en cada ataque, e imprimila al ganar o perder.
- **Otro tema, misma estructura:** este mismo esqueleto (variables + arreglos + `if` + `for` + funciones + input) sirve para una **aventura de texto** (elegís caminos con las flechas), un **selector de pociones** o un **gestor de recursos** tipo estrategia. Cambiás el tema, no la lógica.

Recursos

- Primeros pasos con nodos y escenas (capturas de la Parte A): docs.godotengine.org/es/4.x — **Nodos y escenas**
- Crear tu primer script (capturas de la Parte B): **Creating your first script**
- Fundamentos de GDScript: **GDScript basics**
- Arreglos (`Array`): **referencia de la clase Array**
- Assets gratis (CC0): kenney.nl/assets

Las capturas de este documento provienen de la **documentación oficial de Godot Engine** (Juan Linietsky, Ariel Manzur y la comunidad), bajo licencia **CC BY 4.0**.